Digital Communications and Networks 9 (2023) 492–504

Contents lists available at ScienceDirect

Digital Communications and Networks

journal homepage: www.keaipublishing.com/dcan

Task offloading mechanism based on federated reinforcement learning in mobile edge computing

Jie Li, Zhiping Yang, Xingwei Wang *, Yichao Xia, Shijian Ni

School of Computer Science and Engineering, Northeastern University, Shenyang, 110000, China

ARTICLE INFO

Keywords:
Mobile edge computing
Task offloading
QoS
Deep reinforcement learning
Federated learning

ABSTRACT

With the arrival of 5G, latency-sensitive applications are becoming increasingly diverse. Mobile Edge Computing (MEC) technology has the characteristics of high bandwidth, low latency and low energy consumption, and has attracted much attention among researchers. To improve the Quality of Service (QoS), this study focuses on computation offloading in MEC. We consider the QoS from the perspective of computational cost, dimensional disaster, user privacy and catastrophic forgetting of new users. The QoS model is established based on the delay and energy consumption and is based on DDQN and a Federated Learning (FL) adaptive task offloading algorithm in MEC. The proposed algorithm combines the QoS model and deep reinforcement learning algorithm to obtain an optimal offloading policy according to the local link and node state information in the channel coherence time to address the problem of time-varying transmission channels and reduce the computing energy consumption and task processing delay. To solve the problems of privacy and catastrophic forgetting, we use FL to make distributed use of multiple users' data to obtain the decision model, protect data privacy and improve the model universality. In the process of FL iteration, the communication delay of individual devices is too large, which affects the overall delay cost. Therefore, we adopt a communication delay optimization algorithm based on the unary outlier detection mechanism to reduce the communication delay of FL. The simulation results indicate that compared with existing schemes, the proposed method significantly reduces the computation cost on a device and improves the QoS when handling complex tasks.

1. Introduction

With the rapid development of 5G and wireless communication technology as well as the widespread application of mobile devices, the requirements of mobile users in terms of network service quality are also increasing. According to a market report released by Cisco, the number of devices connected to Internet Protocol (IP) networks will be more than three times the world's population by 2023. The number of network devices per person will increase from 2.4 in 2018 to 3.6 by 2023, and the total number of network devices will increase from 18.4 billion in 2018 to 29.3 billion by 2023. Per capita monthly traffic will be 49.8 GB, and by 2023, the number of global mobile application downloads will reach 299.1 billion [1]. The International Telecommunication Union IMT-2020 [2] defined three main application scenarios in 5G: enhanced Mobile Broadband (eMBB), Massive Machine-Type Communications (MMTC) and ultra-Reliable Low-Latency Communications (uRLLC). uMBB is mainly aimed at high-bandwidth services, such as Virtual Reality (VR)/Augmented Reality (AR) and online 4 K video [3]. MMTC is mainly aimed at the Internet of Things, smart cities, intelligent transportation and other high-connection-density applications. Finally, uRLLC is aimed primarily at time-sensitive services, such as Internet of Vehicles(IoV), Unmanned Driving(UD) and Unmanned Aerial Vehicles(UAV) [4,5]. In the 5G era, performance bottlenecks, such as bandwidth and delay jitter, are difficult to surpass, and 5G communication networks are more decentralised. Therefore, small-scale portable data centres must be deployed at the edge of the network to terminate a large number of

Abbreviations: QoS, Quality of Service; MEC, Mobile Edge Computing; FL, Federated Learning; eMBB, enhanced Mobile Broadband; MMTC, Massive Machine-Type Communications; uRLLC, ultra-Reliable Low-Latency Communications; IoV, Internet of Vehicles; UD, Unmanned Driving; ABR, Adaptive BitRate; DDQN, Double Deep Q-Network; DQN, Deep Q-Network; SNR, Signal to Noise Ratio; DRL, Deep Reinforcement Learning; DNN, Deep Neural Network; KNN, K-Nearest Neighbor; SVM, Support Vector Machine; FIFO, First-In First-Out; mmWave, millimeter Wave.

* Corresponding author.
E-mail addresses: lijie@mail.neu.edu.cn (J. Li), 1871595@stu.neu.edu.cn (Z. Yang), wangxw@mail.neu.edu.cn (X. Wang), 2101825@stu.neu.edu.cn (Y. Xia), nyshij@stumail.neu.edu.cn (S. Ni).

https://doi.org/10.1016/j.dcan.2022.04.006
Received 12 March 2021; Received in revised form 4 April 2022; Accepted 6 April 2022
Available online 11 April 2022
2352-8648/© 2022 Chongqing University of Posts and Telecommunications. Publishing Services by Elsevier B.V. on behalf of KeAi Communications Co. Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

applications. Mobile Edge Computing (MEC) can greatly reduce the application delay to meet the ultra-low latency requirements of uRLLC and MMTC. Moreover, MEC can reduce the bandwidth demand of the transmission network, reduce the transmission cost, improve the efficiency of content distribution, and enhance the user experience. Therefore, MEC is one of the core technologies of 5G. At present, an increasing number of applications are intelligent and must handle a large number of computationally intensive tasks, such as image recognition and rendering in AR/VR. However, due to limited resources, a single device cannot complete the calculation independently, and MEC is influenced by the computing resources of terminal applications. In the field of image recognition, increasing the processing time of the server by 50–100 ms can improve the recognition accuracy by 10%–20%; thus, recognition accuracy can be improved by reducing the transmission delay between the server and mobile terminal by introducing MEC technology to perform computation offloading [6]. MEC uses devices with high processing capacity at the edge of the network to process user tasks, which reduces the network bandwidth consumption and the delay of data transmission and processing [7–9]. With many scenario problems have been improved with the introduction of the edg computing framework, Such as, the problem of video transcoding in Adaptive BitRate (ABR) streaming media and the problem of content caching and user association in edge computing, which involves caching cache popular content in small cell base stations in advance [10,11]. Therefore, MEC-based computing offloading can meet the needs in the eMBB and uRLLC business scenarios. However, the main methods of the computing offloading at present can be divided into two categories: optimization algorithms and heuristic algorithms. Owing to the time complexity of task assignment and collaborative programming, a large amount of time is required to use an optimization algorithm to find the optimal solution for a large-scale MEC system. Although heuristic algorithms offer some improvement, because they must obtain the optimal solution in a certain time, the solution is often the local optimal solution rather than the global optimal solution. To solve this problem, we propose a dynamic and adaptive offloading decision algorithm. The main contributions of this study are as follows:

(1) For a single-user multi-tasking device, policy modification is performed by introducing the Double Deep Q-Network (DDQN) algorithm and obtaining an offloading policy model through training. The policy model can provide an offloading policy according to the channel conditions of the current user. Additionally, it can solve the problem in which dynamic changes in channel conditions affect the offloading policy.
(2) We adopt a distributed training method in which the learning model is deployed on each device, and each learning model is only responsible for the task decision of its user. Because the local device data may be limited, the decision model for multiple devices adopts Federated Learning (FL) optimization to reduce the decision-making model of personalised features. Moreover, FL is used to obtain a more generalised global model, allow new users to directly use the model on the basis of the training model, and reduce the decision-making model training time of new users.
(3) For the learning of each device, because link states cause different transmission delays in transporting parameters and performing global iterations, a delay optimization algorithm is proposed. The algorithm uses a univariate outlier detection method to detect and screen the transmission delay of all users in each global iteration, and cancel the participation qualification of users whose Contribution Rate is unqualified. According to this screening method, the transmission delay of each iteration is selected, the delay generated by the federated framework is optimized, and the advantages of FL are fully used to improve the quality of user experience.
(4) The results of simulation experiments using the proposed algorithm demonstrate that under dynamic channel changes, the proposed algorithm can perform appropriate offloading decision-making and improve the quality of experience.

2. Related work

In MEC, task offloading has attracted extensive attention in academia and industry. In Ref. [12], a fast heuristic algorithm was proposed to obtain a solution satisfying various constraints. To minimise the system cost, Guo et al. proposed an intelligent offloading strategy based on the ant colony algorithm, in which ants randomly access the upper limit to obtain the final offloading results [13]. Yang et al. proposed an online energy-saving task allocation and computation offloading strategy based on dynamic wireless conditions and service delay constraints to coordinate and optimise wireless and computing resource allocation [14]. Furthermore, Sun et al. studied parallel offloading of multi-tasking applications to MEC to minimise the completion time [15]. For the scenario in which the edge server cluster serves mobile users, a two-layer architecture was considered for maximising the quality of service [16]. In another study, Josilo et al. modelled the interaction between a device and operator as a Stackelberg game and proposed an efficient decentralised algorithm for calculating equilibrium [17]. An online algorithm, DEDAS, was proposed in Ref. [18], which greedily schedules newly arriving tasks and determines whether existing tasks should be replaced to meet the new deadline. In Ref. [19], based on the recent development of continuous convex approximation, an energy-saving computing resource allocation scheme was proposed, and the optimal solution was obtained by a careful study of the problem structure. In other research, Pasteris et al. proposed a deterministic approximation algorithm to solve the problem of reward maximisation in MEC networks [20]. Sundar and others designed an algorithm that greedily optimizes task scheduling based on the time of each task in the allowed range to solve the deadline allocation problem [21]. In Ref. [22], the authors proposed a reinforcement-learning-based state-action-reward-state-action algorithm to solve the problem of edge server resource management. This approach involves optimising the uninstall decision to minimise the system cost, including the energy consumption and calculation time delay.

In a research setting, channel conditions are sufficiently stable to establish a mathematical model of the problem and obtain the optimal solution. In practice, however, the channel state changes dynamically with uncontrollable environmental factors. By using traditional methods that do not take into account the dynamic changes in channel conditions [23], real-time decision-making under a dynamic workload and wireless environment is impossible. At present, Deep Q-Network (DQN) and Q-learning are used in various solutions to address dynamic changes in channel conditions. Q-learning is appropriate for decision-making in scenarios with few feature dimensions, i. e, there are few state spaces and action spaces in such scenarios. However, in real-life scenarios, decision-making problems have huge state and action spaces. To build a Q table, memory is not allowed, and the amount of data and time overhead are both big issues, we call this the dimensional disaster problem. On the one hand, Q-learning is not suitable for such scenarios whose state space is too large, and is likely to cause dimensional disaster [24–27]. On the other hand, DQN tends to cause an overestimation of the experimental results.

In multi-user scenarios, previous studies used a learning model to make decisions for user tasks in the DQN framework. This learning model centrally learns all terminal device data, and when private data are involved, this approach leads to leakage of user data in practice. Traditional machine learning methods centralise all data in one location, usually a data centre, which is likely to violate user privacy and data confidentiality laws [28]. FL is a distributed learning method proposed by Google to protect user data privacy. It allows each terminal load to cooperate without exchanging data, improves the performance of machine learning, and solves the problem of data islands and data privacy [29]. In contrast to traditional centralised learning, FL is a common conceptual model that is equivalent to the optimal model for learning by

aggregating data; however, the local data of individual users are not migrated to protect privacy. The obtained common model serves only individual users to achieve their respective goals [30,31]. In contrast to existing methods, our proposed method assumes that users may refuse to disclose their network bandwidth information and other private data.

In view of the above-mentioned challenges, we propose a dynamic adaptive offloading algorithm. Taking into account the limitations of Q-learning and DQN algorithms, we use the DDQN algorithm to obtain the task decision model on the local device. The DDQN algorithm is introduced into computation offloading in a single-user multi-tasking environment of a MEC system to generate the most appropriate offloading policy under dynamic channel conditions. Considering that terminal users attach importance to their data privacy in real life, and the problem of a single terminal with limited datasets, we use FL to optimise the decision model for multiple users in a distributed manner.

3. System model

The framework considered in this study is a hierarchical architecture consisting of N devices, an access point and a MEC server. The framework maps two layers, namely, the device layer and edge layer, as illustrated in Fig. 1. In real life, mobile users consider only personal benefit maximisation. The main consideration is in the framework of a single device on multiple tasks are part of the offloading. As in the literature [32], we ignore the correlation between tasks, assuming the same equipment and no correlation between produce multiple parallelism tasks. For example, multiple applications running at the same time generate multiple independent different calculations, and AR applications involve many task-dependent processes; however, these tasks can be divided to increase task parallelism [33,34]. Each task on a mobile device can be executed locally or migrated to a MEC server over a wireless network for processing, depending on the current environment, to expand the computing power and reduce the power consumption of the mobile device. The processing results are then returned. A wireless link is used to transmit data between the mobile device and MEC server. The system model is presented in Fig. 1.

3.1. Task model

Assume that there are N tasks on the mobile device, denoted by set $N = \{1, 2, \ldots, i, \ldots, n\}$.Each task can be defined as a triple $Task_n = (d_n, c_n, t_n^{max})$, where d_n represents the data size of the computing task, c_n represents the number of Central Processing Unit (CPU) cycles required to complete the task $Task_n$(i.e., the computing resources required for the computing $Task_n$), t_n^{max} is the maximum tolerated delay for $Task_n$. The offloading policy on mobile devices is $A = \{a_1, a_2, \ldots a_n\}$. A represents the computational state of N independent tasks. When $a_i = 1$, the task is computed on the MEC server. When $a_i = 0$, the task is executed locally. Thus, the offloading policy is a set of elements 0 and 1.

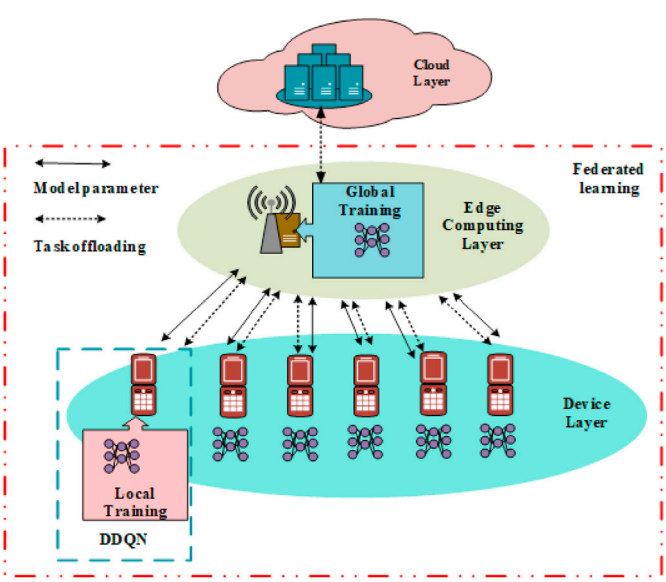

Fig. 1. Task offloading in mobile edge computing.

3.2. Local computing model

When a task is processed locally, the delay is the processing delay of the task on the CPU. f_i^l represents the computation frequency locally assigned to $Task_i$. The local computation delay is defined as

$$T_i^l = \frac{c_i}{f_i^l} \tag{1}$$

where c_i represents the number of CPU cycles required to complete $Task_i$. The local energy consumption is defined as

$$E_i^l = \delta (f_i^l)^2 c_i \tag{2}$$

where δ represents the energy consumption coefficient of each cycle. The cost of local computation is:

$$Q^l = \sum\nolimits_{i=1}^{n} (\alpha T_i^l + \beta E_i^l)(1 - a_i) \tag{3}$$

where Q^l represents the QoS generated by computing locally, α and β represent the weight of the delay and energy consumption, respectively. The weight value remains the same each time the policy is executed.

3.3. Edge computing model

When offloading a task to a MEC server for processing, the data are first transmitted to the base station and then forwarded from the base station to the MEC server. This study focuses on the latency and energy consumption from the user's viewpoint. MEC servers are typically powered by the grid, therefore, we ignore the energy calculation on the MEC server side [35,36]. Finally, the server returns the calculation and processing results to the terminal device in the form of data. Because the size of the uploaded data is significantly larger than the size of the data returned from the server and the download rate is higher than the upload rate, we ignore the transmission delay in the process of returning the calculation results to the user device [37,38]. The delay is defined as

$$T_i^s = \frac{d_i}{r} \tag{4}$$

where r represents the upload rate of the network channel (as a function of channel gain h),

$$r = B\log_2(1 + \frac{hp}{I}) \tag{5}$$

where B represents the transmission bandwidth, p represents the transmitted power, h represents the channel gain, and I represents the Signal to Noise Ratio(SNR). The corresponding energy consumption during the task offloading process is defined as

$$E_i^s = pT_i^s = \frac{pd_i}{r} \tag{6}$$

The cost of edge computation is:

$$Q^s = \sum\nolimits_{i=1}^{n} (\alpha T_i^s + \beta E_i^s) a_i \tag{7}$$

In sum, the total system consumption (the Quality of Service, QoS) is

$$Q = \sum\nolimits_{i=1}^{n} (\alpha \frac{c_i}{f_i} + \beta E_i^l)(1 - a_i) + \sum\nolimits_{i=1}^{n} (\alpha T_i^s + \beta E_i^s) a_i \tag{8}$$

Our goal is to minimise the total execution delay and energy consumption for all tasks on terminal devices in the MEC system, that is, to minimise Q by performing local task offloading.

$$\min Q = \min \sum_{i=1}^{n} (\alpha T_i^l + \delta (f_i^l)^2 c_i)(1 - a_i) + \sum_{i=1}^{n} (\alpha \frac{d_i}{r} + \beta \frac{pd_i}{r}) a_i \tag{9}$$

subject to the following conditions:

$$C1 : T_i^s <= t_i^{max} \tag{10}$$

$$C2 : E_i^l <= E_i^{max} \tag{11}$$

$$C3 : \sum f_i^l <= f \tag{12}$$

$$C4 : 0 < \delta < 1 \tag{13}$$

$$C5 : \alpha + \beta = 1, (0 < \alpha < 1, 0 < \beta < 1) \tag{14}$$

$$C6 : a_i \in \{0, 1\} \tag{15}$$

where t_i^{max} is the maximum acceptable delay and E_i^{max} is the maximum acceptable energy consumption. Because this problem is solved by determining the optimal value of the decision vector $A = \{a_1, a_2, \ldots a_n\}$ under multiple constraints, the complexity of the problem increases with an increase in the number of tasks, thus, it is an NP-hard problem.

4. Optimal offloading policy algorithm

4.1. Scheduling decisions based on deep reinforcement Learning(DRL)

For users with N tasks we have an offloading policy matrix P, which includes all possible offloading scenarios. The rows of the matrix represent the number of decisions, while the columns of the matrix represent the individual tasks $P = \begin{Bmatrix} 1 & 0 & \ldots & 1 \\ 0 & 1 & \ldots & 0 \\ \ldots & \ldots & \ldots & \ldots \\ 1 & 1 & \ldots & 1 \end{Bmatrix}$. We can observe from the matrix P, that the total number of decisions for N tasks in each time frame is 2^N. In a short time frame, we must find a decision within the 2^N possible decisions. The computational complexity and time consumption increase exponentially with the number of tasks. Therefore, traditional value-function reinforcement learning is not feasible. Consequently, we propose using DRL for learning offloading scheduling within a given time frame. When DRL is used to make decisions, the state is defined as the size and number of tasks, the current channel gain and the locally available CPU processing capacity of the task. The DRL agent outputs the corresponding action according to the input of the states, namely, the task scheduling decision.

First, we study the computation offloading of multiple tasks independent of each other on a single device. We use the binary (0–1) paradigm to represent the offloading of each task. The offloading policy for multi-tasking is a set of 1 and 0, whose number represents a task. Considering that the link state is time-varying, we divide the system time into multiple time frames, smaller than the channel coherence time, using time division multiple access. In this study, we use DDQN to implement real-time intelligent decision-making services for equipment. Then, for policy-making, we use DDQN to continuously learn the experience value to obtain a learning model that can generate a offloading policy for multiple tasks according to the current channel state. In DDQN, we use the normal distribution to initialise the network parameters in Deep Neural Network(DNN).

During the execution of the algorithm, the output of the algorithm should be quantified and classified. Typical classification methods include K-Nearest Neighbor(KNN), logistic regression, decision tree, naive Bayes, and Support Vector Machine (SVM). Among them, the decision tree is unstable, and small changes in data may result in a completely different tree; Naive Bayes must define a prior probability in advance, but a prior model defined in advance is hypothetical, which will cause poor overall prediction results; SVM is a supervised learning method, that can be used in complex scenarios. However, the classification scenario in this study is very simple, so SVM does not meet the application scenario requirements; Conversely, because the KNN proximity algorithm is simple and effective, it is suitable for automatic classification of data in this scenario. It does not require training data and the training complexity is low. The output result of the KNN algorithm is very simple, and the amount of data is small. In summary, in this scenario, the output classification of the network is very simple and effective, and the KNN algorithm is more suitable than other methods.

When the KNN algorithm is used, the selection of the K value has a significant impact on the results of the algorithm; thus, we use cross-validation to select the optimal K value. The output of the network is quantified and classified using the KNN algorithm to obtain a decision set comprising 0 and 1. Subsequently, the corresponding Q value is obtained according to the action which is an Offloading decision. In the decision matrix, the Q value representing QoS, and greedy algorithm are used to filter the decision results of multiple tasks in the current time frame. The results are filtered to obtain the optimal result as output.

Because the intelligent decision model designed in this study is obtained through DDQN learning and training based on continuous time, the time series is defined as $T = \{1, 2, \ldots t, \ldots, T\}$.Three key DDQN factors in the edge computing offloading decision-making environment are as follows: State, Action and Rewared. They are defined as follows:(1) State: The system status is divided into three parts: the size of each task collected in each time slice in each device, the number of tasks, and the current network channel gain, which can be expressed as $s_t = \{(d_1, \ldots d_N), N, h_t\}$. The entire state space is based on continuous time, so the setting is $S = \{s_1, s_2, \ldots, s_t, \ldots, s_T\}$; (2) Action: Obtain the action set according to the current state. Based on the reward value, a greedy algorithm is used to select an optimal result $A_t = \{a_1, a_2, \ldots, a_n, \ldots, a_N\}$ as the task offloading decision at that moment. After executing the task decision, the device will transition from the current state s_t to the next state s_{t+1}. The entire action space is also based on continuous time, so it is set to $A = \{A_1, A_2, \ldots, A_T\}$; (3) Reward: When the DRL agent on the device interacts with the environment to obtain the states$_t$, and then s_t will be input into the main network to obtain the corresponding action value, and then the reward value will be used to measure the quality of the action selected. There will be one for each decision return value.

Local memory is allocated to store the optimal policy to store the current state (h_t, $Data$, N)(where $Data$ is a set of task sizes, $Data = \{d_1, d_2, \ldots, d_N\}$), action A_t, reward value r_t, and next moment state s_{t+1} obtained from the interaction between the action and environment in the current time frame. These data (s_t, A_t, r_t, s_{t+1}) are stored in the newly allocated memory, and are used for the next training of the learning model. The process of using the experience value for model training is as follows: input s_t and s_{t+1} in memory into the original network to obtain the value of the function and the optimal Q value in the next step using the following equation:

$$A_{t+1} = \arg\max_{A_t} Q_{Main}(s_{t+1}, A_{t+1}, w_{main}) \tag{16}$$

Next, the value function of A_t is input into the target network to obtain $Q_{target}(A_{t+1})$. Then the loss based on the loss function is calculated:

$$Loss = E\left[\left(r_t + \gamma Q_{target}\left(s_{t+1}, A_{t+1}, w_{target}\right) - Q_{Main}(s_t, A_t, w_{main})\right)^2\right] \tag{17}$$

Then the loss is transmitted back to the main network, and the network parameters are updated. And after a period of time, the target network will copy the model parameters of the main network to update the model. The flowchart of experience play based on DDQN is as follows:

According to the Behrman equation, the current target Q value q_t can be calculated from the pre-estimate of r_t and A_{t+1}; as follows (see Fig. 2):

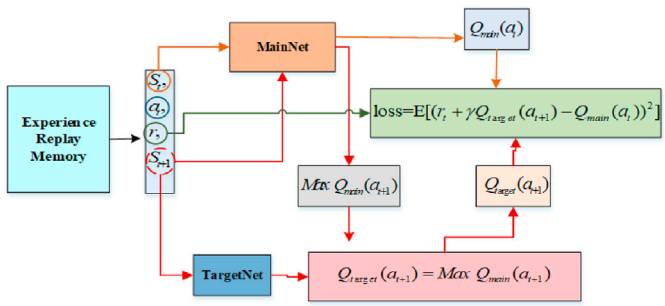

Fig. 2. The flowchart of experience play based on DDQN.

$$q_t = r_t + \gamma Q_{target}(s_{t+1}, A_{t+1}, w_{target}) \tag{18}$$

$$q_t = r_t + \gamma Q_{target}(s_{t+1}, argmax Q_{Main}(s_{t+1}, A_{t+1}, w_{Main}), w_{target}) \tag{19}$$

The network parameters of the original network are updated as

$$w_{Main} = w_{Main} + \zeta(Q_{Main}(s_t, A_t, w_{main}) - q_t) \tag{20}$$

where γ and ζ are hyperparameters. When the memory is full, the principle of First-In First-Out (FIFO) is adopted to replace old data with new data, which will improve the quality of the training data. A schematic of the process is presented in Fig. 3.

4.2. Policy model optimization for FL

Challenges to address include the limited dataset of the task model on a single device, the privacy of local data, and the individual applicability of the local model of each device. We use FL to optimise the model to reduce the personalised characteristics of each local device model, improve the universality of the model, and be able to directly apply the model to new users, reducing the model training time for new users. Additionally, the distributed use of data on multiple local devices solves the problem of a limited dataset. Further, applying distributed training does not require uploading all data for centralised training, which protects data privacy.

FL is divided into two processing modes: local and global parameter updates. In the local update, a single device continues to train the local model on the basis of global processing parameters w_{global}. In the global update, multiple local devices offload their model parameters to the server. Here, $user = \{1, 2, \ldots, j, \ldots, U\}$ represents a set of users. The server processes these parameters w_j uniformly, as in equation (23), where w_j represents the model parameter of user j. The global loss function $F_{global}(w)$ expressed as equation (24), where D denotes the dataset of all users, D_j denotes the data set of user$j$,F_{global} is the loss function of the global model, F_j is the loss function of user j, w_{final} is the final global model parameters obtained by training. Then w_{final} is returned to each mobile terminal.

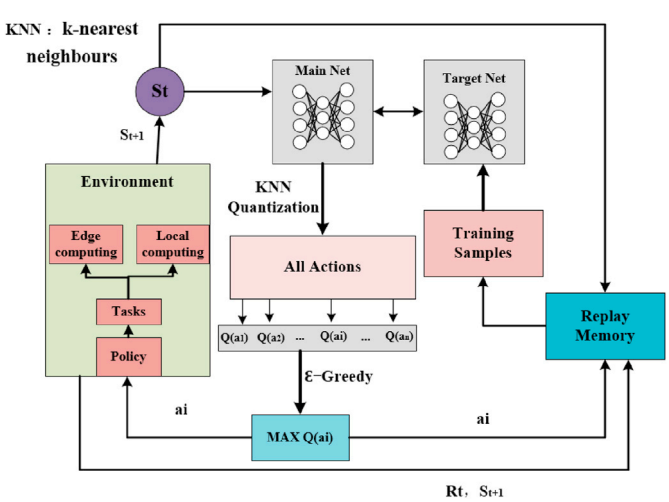

Fig. 3. Flowchart of DDQN based task offloading policy.

$$D = \sum_{j}^{U} D_j \tag{21}$$

$$\rho_j = \frac{D_j}{D} \tag{22}$$

$$w_{global} = \frac{\sum_{j=1}^{U} D_j w_j}{D} \tag{23}$$

$$F_{global}(w) = \frac{\sum_{j=1}^{U} D_j F_j(w_j)}{D} \tag{24}$$

$$w_{final} = \arg\min \frac{\sum_{j=1}^{U} D_j F_j(w_{global})}{D} \tag{25}$$

The local and global updates are repeated in the system time to satisfy equation (25). A schematic of this process is shown in Fig. 4.

When DDQN is used for local training, the user and server are in a one-to-one mode. With the introduction of FL, the user and server are in a one-to-many relationship. The Offloading policy model generation is illustrated in Algorithm 1. FL consumes a certain amount of resources in the process of obtaining the global model. This study mainly focuses on the delay and energy consumption of this process. Because the processing capacity of the server is significantly greater than that of the local device, the processing delay of the server can be ignored. It mainly includes the delay of local model iteration and the delay of uploading local model parameters. The delay model for global processing is as follows:

$$T_{global} = T_{cmp} + v\log(1/\theta)T_{com} \tag{26}$$

where v represents a constant related to the size of the local transmission parameter, θ is the convergence degree of the local model, T_{com} is the transmission time($T_{com} = argmax\{T^1_{com}, \ldots, T^i_{com}, \ldots\}$), and $T^i_{com} = d(w_i)/r_i$, where $d(w_i)$ is the size of w_i, and r_i is the transmission rate. T_{cmp} is the local computation time. The energy consumption includes the energy consumption of local model training and the transmission energy consumption of the uploaded model parameters. The energy consumption model of global processing is as follows:

$$E_{global} = \sum_{n=1}^{N} (E_n^{com}(\tau_n) + E_n^{cmp}(f_n)) \tag{27}$$

The energy consumption of the local iteration model is

$$E_n^{cmp}(f_n) = \sum_{i=1}^{c_n D_n} \frac{\alpha_n}{2} f_n^2 \tag{28}$$

where f_n is the computing frequency of the user; D_n is the amount of data that user n is involved in learning; c_n is the number of CPU cycle required for sample data processing; $\frac{\alpha_n}{2}$ is the capacitance coefficient.

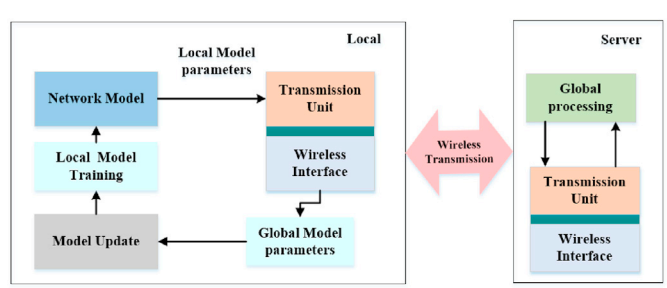

Fig. 4. Model optimization framework based on FL.

Algorithm 1 Adaptive task offloading policy algorithm based on DRL and FL.

Input:
Global update interval φ,target-Q-network update frequency ϕ in DDQN,Number of participating users U;
Output:
Task offloading policy A ;
1: Initialise w_{target}, w_{Main}, s,Q and memory size M;
2: Set the DNN training interval *train* and learning rate lr.
3: **for** t to T **do**
4: Perform steps 5 to steps 19 for U local users at the same time.
5: Obtain current system status s_t: Channel gain h_t in each time frame, number of tasks N, size of task $\{d_1, d_2, ..., d_N\}$.
6: s_t as input in main-Q-network to obtain main-Q-network output *out*;
7: KNN algorithm quantise *out*, obtains the binary policy;
8: Substitute all decisions into equation (9) to calculate Q_t ,
9: $\epsilon - greedy$ is used to select the optimal action A_t and output it;
10: Obtain the current action A_t in state s_t, and obtain a new state s_{t+1} and reward r_t ;
11: Store (s_t, A_t, r_t, s_{t+1}) in memory ;
12: s_t=s_{t+1};
13: **if** $Memory = M$ **then**
14: Replace old data with new data by FIFO algorithm;
15: **end if**
16: Sample m samples from the experience memory, and calculate the current target Q value q_t according to equation(18);
17: Use the mean square error loss function$\frac{1}{m}\sum(q_t - Q_{Main}(s_t, A_t, w^t_{Main}))$ to update w^t_{Main};
18: **if** $t\%\phi = 0$ **then**
19: $w^t_{target} = w^t_{Main}$;
20: **end if**
21: **if** $t\%\varphi = 0$ and w_{final} is not convergence **then**
22: Upload all users' w^t_{Main} to the federated agent;
23: Obtain w_{global} according to equation(23);
24: For all users: $w_j = w_{global}$, $j = 1, 2, ..., U$;
25: Update w_{final} according to equation(25);
26: **end if**
27: **end for**
28: Obtain $Model_{final}$
29: Obtain the current state S and as the input of $Model_{final}$ to obtain the task offloading policy A
30: **return** task offloading policy A;

The transmission energy consumption model is as follows:

$$E_n^{com}(\tau_n) = \tau_n p_n = \frac{d_n}{r_n} p_n \tag{29}$$

where d_n denotes the size of the parameter transmitted by user n, τ_n denotes the transmission time of user n, r_n denotes the transmission rate of user n, and p_n denotes the transmission power of user n.

4.3. Communication delay optimization algorithm based on unary outlier detection

In each iteration of FL, it is necessary to wait until the parameters of all participating users are successfully uploaded before performing federated processing. The main delay cost is the delay incurred by the last user who upload the parameters to the federated agent. In the FL framework, the difference in the link quality between the device configuration parameters of participating users and the channel to which they belong leads to different transmission delays when uploading the parameters. During each federated iteration, individual participating users cause excessive transmission delay due to link quality, which makes the federation processing delay relatively large. Therefore, during the federation process, we consider removing few participating users whose time delay is much larger than that of other users.

In this study, users whose communication delay is significantly larger than that of other participating users are called outliers. In the process of FL, there will be many iterations, and such outliers will be generated uncontrollably in each iteration. The delay superposition caused by outliers in the iteration process will seriously increase the delay cost of FL framework, reduce the quality of the model, increase the number of iterations, and reduce the quality of user experience. As outliers are only determined by the delay, it is one-dimensional outlier detection, also known as unary outlier detection. Because outliers exists in a dataset as abnormal data Points, the probability of their occurrence in data is very low. Therefore, the method of detecting outliers in a set of data can be based on probability, and the data with lower probability are regarded as outliers. However, this method also has obvious disadvantages. Although these data are low-probability objects, they are also have other important Functions.

To compensate for the disadvantages of the method, the Contribution Rate(refers to the degree of influence on the overall model) is used to perform detection on the selected outliers to prevent false detection results in this study. This is a statistics-based approach: It is assumed that normal data objects are produced using a statistical model (such as a normal distribution), and data that do not conform to that model is an outlier. The outlier detection method for this distribution depends on the distribution of data, parameters (such as mean or variance), and expectations (confidence interval). Statistical methods for outlier detection based on standard statistical techniques such as estimation of distribution parameters, so it has a solid foundation, Most statistical methods are used to process the single attribute, that is, the judgment of one-dimensional space. The communication delay set is one-dimensional, so this article considers using a normal distribution-based unary outlier detection method to detect outliers in the delay set.

Algorithm 2 Communication delay optimisation algorithm based on unary outlier detection.

Input:
Model parameter size of all users $\{d(w_0), d(w_1), ..., d(w_j), ..., d(w_U)\}$;
Output:
Transmission delay of this iteration T_{com} ;
Obtain transmission delay set $T_{com}^{set} = \{T_{com}^1, ..., T_{com}^j, ..., T_{com}^U\}$according to $T_{com}^j = \frac{d(w_j)}{r_j}$;
Calculate the mean of the delay set:$T_{aver} = \frac{\sum_{j=1}^{U} T_{com}^j}{U}$;
Calculate the variance of the delay set:$\sigma^2 = \frac{\sum_{j=1}^{U}(T_{com}^j - T_{aver})}{U}$;
Find outlier set T_{com}^{out} beyond the region of $T_{aver} \pm 3\sigma$;
if $T_{com}^{out} = NULL$ **then**
  $T_{com} = \arg\max T_{com}^{set}$;
else
  Record the total number of collection elements *number* in transmission delay set;
  for $T_{com}^j \in T_{com}^{out}$ **do**
    if $\rho_j > 10\%$ **then**
      Retain the user's qualification to participate
    else
      Cancel the user's participation qualification,and remove elements T_{com}^j from the collectionT_{com}^{set} ;
      $number = number - 1$;
      if $number == 0$ **then**
        $T_{com} = \arg\max T_{com}^{set}$;
      end if
    end if
  end for
end if
return T_{com};

We also need to judge its contribution rate of outliers. If the contribution rate is high, we need to continue to retain the participation of outliers. Because the high contribution rate will have a significant impact on the global model, If the outlier participation qualification is hurriedly cancelled, the number of federation aggregations will increase, which will increase local resources and delays. Cancelling the participation qualification of the outlier also means rejecting the participating user to participate in this federated iteration aggregation. Therefore, the qualification of an outlier needs to be judged using both the outlier detection algorithm and the contribution rate at simultaneously. This study uses an experimental method to verify the influence of the contribution rate on the federal framework, as shown in Table 1. The table shows that when outliers with a contribution rate of less than 5% and 10% are disqualified, the resource consumption of the overall framework and the quality of the model are affected and have less impact on the resource consumption of the overall framework. However, when the contribution rate exceeds 15%, the impact on the resource consumption of the overall framework and the quality of the model is significant. Based on the analysis of Table 1, when the contribution rate of the detected outlier is less than 10%, it can be considered to be disqualified from participating. The specific processing flow is shown in Fig. 5.

Based on the above method, after judging all the outliers, select the largest delay from the delay set of the remaining participating users as the communication delay of this federation iteration. This step is used in each iteration, and finally communication delays are accumulated and summed at the end of the FL. Here, we use a detection algorithm based on a unary outlier. The specific algorithm is presented in Algorithm 2.

5. Simulation experiment

To evaluate the performance of the proposed algorithm, we considered a typical MEC task offloading scheduling scenario. The current processing capacity of a typical mobile processor CPU is $f_i^l \in [0.5Ghz, 1.3Ghz]$. In communication transmission, the 5G working frequency is divided into FR1 and FR2. Among them, FR1 represents the low-frequency band, also known as centimeter wave (sub6G), the channel bandwidth can reach up to 100Mhz; FR2 represents the high-frequency band, also known as millimeter Wave (mmWave), the maximum channel bandwidth can reach 400Mhz. Although 5G technology is divided into two communication frequency bands, sub6G is still the mainstream in the world, so in this scenario, the channel bandwidth in the normal communication link is set to 100Mhz; According to the regulations of the WIFI Alliance, the standard transmission power of a wireless router should be less than 100 MW regardless of its type; therefore, we set the wireless transmission power p to 0.1W. In the experiment, the number of cycles required by each task was uniformly distributed as $c_i \in [1000, 1300]$. Similarly, the data size of each task obeyed a uniform distribution $d_i \in [100\ kb, 400\ kb]$. The SNR of the communication link in the wireless transmission process was set to $I = 1.43 \times 10^{-6}$, and the cycle energy consumption coefficient of the local CPU was $\delta = 10^{-25}$. The weight parameters of delay and energy consumption were set to $\alpha = 0.4$ and $\beta = 0.6$,respectively. The TensorFlow and Keras frameworks were combined to conduct experiments and training for the proposed neural network.

We first conducts experiments on the model loss of the model and performed DRL model training based on 20,000 state data. We stored the optimal Q value and state data in memory; then, the data were transferred to the DNN for subsequent learning and testing of the decision model. First, we set the number of network layers to four layers and compared the model loss under different learning rates to determine the training loss change, as illustrated in Fig. 6. It can be seen that the model loss was not ideal when the learning rate was 0.1 and 0.0001. In contrast, when the learning rate was 0.1 and 0.0001, the training loss was relatively high, whereas when the learning rate was 0.01, the loss decreased earlier. Based on the four-layer network architecture, the optimal learning rate is 0.01. As shown in Table 2, a decrease in the learning rate within a certain range can increase the model loss, and beyond this range, the model loss may also decrease because of a decrease in the learning rate.

To determine the optimal number of network layers and keep other parameters unchanged, we increased the number of layers of the network to six and continued to compare the training loss under different learning rates. The changes in the time frame are illustrated in Fig. 7. Fig. 7 indicates that the difference between the best model loss of the six-layer network structure and that of the four-layer network structure was very small. From Table 2, we can also see that the number of network layers greatly increased. Therefore, the number of network layers was not proportional to the model loss, and the increase in the number of network layers increased the training time. Moreover, under the six-layer network

Table 1
The effect of the contribution rate of disqualified outliers on the framework and the global model.

CR	Delay(s)	Iterations	Loss	Energy cost(J)
5%	301.4865	25	0.15306	16.4867
10%	317.0325	23	0.16012	15.3278
15%	379.439	31	0.16154	20.5329

structure, when the learning rate was 0.001, the model loss was the highest. It can thus be seen that a difference in network structure can cause differences in the optimal learning rate.

Based on the above analysis, in the network structure composed of a DNN, we set the number of network layers to four, namely, the input layer, first hidden layer, second hidden layer and output layer, and set the learning rate to 0.01. The activation function was the sigmoid function.

Fig. 8 illustrates the effect of the memory size used to store the results and state values, while Fig. 9 illustrates the effect of the batch size used for DDQN training on the accuracy of the decision model. Fig. 8 indicates that larger memory did not lead to improved model training. This is because larger memory led to storing more historical data, which led the data update to not be timely. The data were selected for the model each batch during training, which led to a high probability of selecting old data stored for a long time, thereby affecting the accuracy of model training. Similarly, the smaller the memory, the more limited the range of batch sizes used for model training. There were thus insufficient historical data for model training, resulting in a decrease in model training accuracy. In Fig. 9 we can observe that the batch data for each training is not the largest and the best, because the batch data is too large and the training time increases, the local optimization is poor, and the generalization is poor. The batch data is too small to converge. So according to Figs. 8 and 9, we choose the appropriate size of memory space and batch size.

To verify the overall performance of the DDQN-based adaptive offloading decision model algorithm on the local terminal, the algorithm was compared and verified with other algorithms. There are two comparison algorithms, they are as follows: (1) RandomOffloading: all tasks are calculated randomly on the local or edge server; (2) Policy-DQN: The DQN algorithm is used to train the learning decision model. When the model is stable, the computing task is decided and scheduled. The experimental results are shown in Fig. 10. The RandomOffloading offloading decision-making algorithm is also a combination of local and edge methods to deal with task calculation problems. Fig. 10, that the RandomOffloading offloading decision-making algorithm has the largest overhead, and during the experiment, its overhead fluctuates greatly. This is because it does not systematically inspect the overall situation for uninstallation but uses a random uninstall method. Therefore, in some cases, the unsatisfactory link quality will cause the transmission calculation to be untimely, and the overall overhead will be too large. However when the link quality is ideal, the combination of local and edge servers will perform better. Compared with RandomOffloading, the offloading decision algorithm ourPolicy in this study will fully consider the local calculation situation and the quality of the link to make a decision, to present an offloading scheme with the least overall overhead in the current state. Policy-DQN is similar to the algorithm in this article, but because the Policy-DQN algorithm is a model obtained by using DQN, its decision result is overestimation, making in the task scheduling decision given by the decision model is prone to bias and increase the overall cost

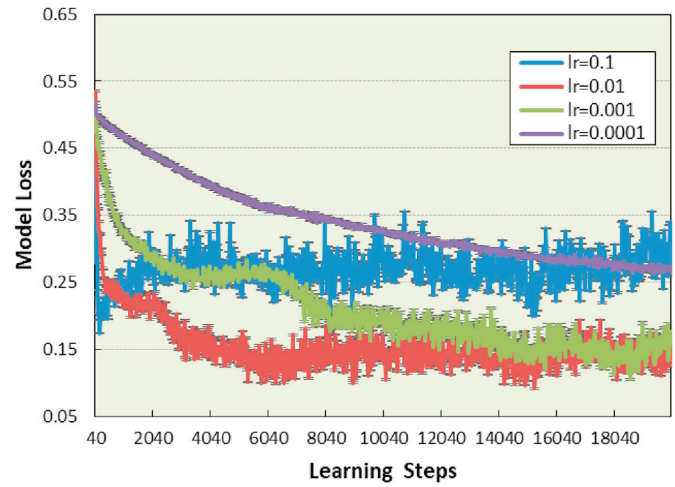

Fig. 6. Losses of different learning rates in a four-layer network structure.

of the task. Compared with Policy-DQN, the decision model obtained using ourPolicy could yield more appropriate computing task scheduling decisions based on the current computing environment state, which can minimise overhead costs and improve the quality of user experience. Experimental results show that our decision algorithm is better than the other two offloading decision algorithms in terms of overhead cost.

To determine the impact of FL on the decision model and the number of participating users, we first set the number of participating users under the FL framework to 30, 20, 12 and 8, respectively, to conduct the iterative experiment of the global model. Fig. 11 indicates that a different number of participating users resulted in a similar trend. With few iterations, the model loss was larger, whereas with more iterations, the results gradually stabilised. Fig. 11 indicates that with fewer iterations, the model quality was affected by the number of users: the more users, the higher the model quality. When the number of global iterations reached approximately 30, the final global decision model was obtained. The quality of the global model with 30 participating users did not continue to improve, and the quality of the final model was close to that of the model with 20 and 12 users. The number of users had no significant effect on the quality of the model.

The above analysis indicates that at the early stage of the experiment on the entire training data set, the results were affected by the number of users. However, in the later stage, when a certain number of iterations was reached, the quality of the overall model was stable and the number of users did not affect the quality of the model. The loss interval of the global model was within (0.1, 0.2), which was not different from the model quality without FL. We can also see from Fig. 11 that the more participating users there were, the more distributed the data sets were

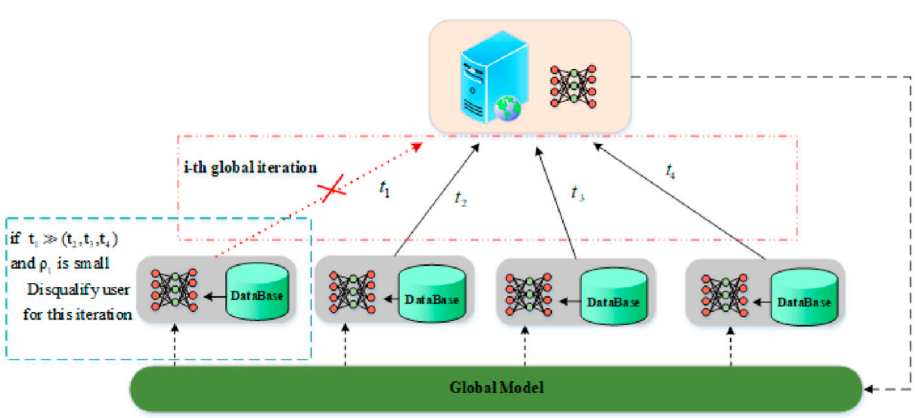

Fig. 5. The workflow of communication delay optimization algorithm for FL.

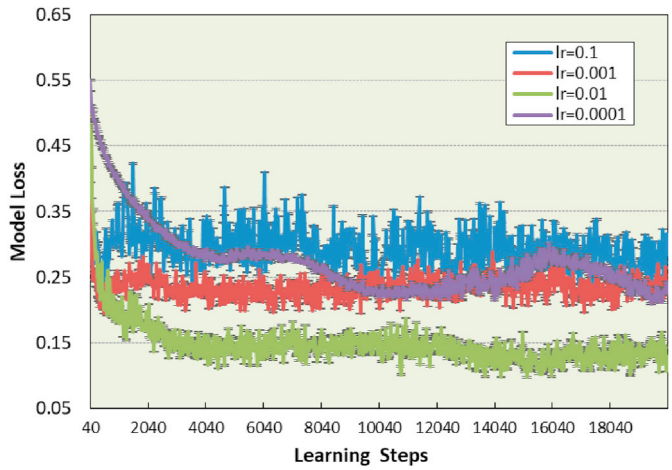

Fig. 7. Losses of different learning rates in a six-layer network structure.

Table 2
Model training time under a different number of network layers and different learning rates.

Learning rate	Four layers (s)	Six layers (s)
lr = 0.0001	143.5828	351.235
lr = 0.001	180.0455	330.415
lr = 0.01	131.0742	344.7119
lr = 0.1	138.3382	360.6051

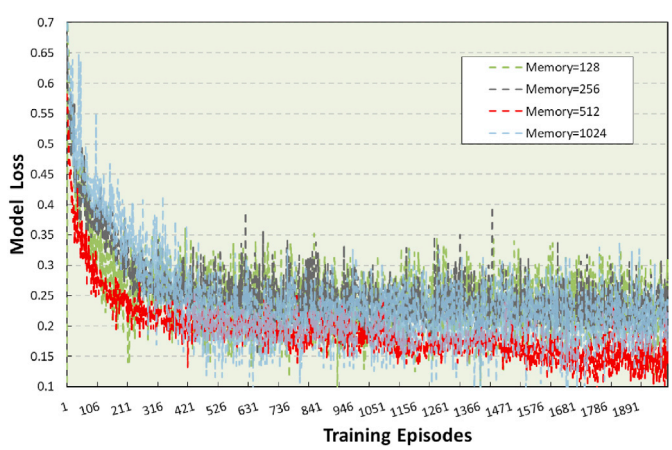

Fig. 8. Effect of memory size on model loss.

and the faster the model stabilised.

In the process of introducing FL, the data sets of each terminal are used in a distributed manner. Compared with centralized training, distributed training will cause additional delay and energy consumption of model parameter transmission. So the next step is to verify the resource consumption of the device in the process of obtaining the global decision model after the introduction of the federated framework, that is, the energy consumption and delay of model training when there is FL and No-FL influences. Simultaneously, it will continue to explore the impact of the number of participating users in the federal framework on energy consumption and latency.

Due to the huge difference in value between time delay and energy consumption, if the QoS model is used to calculate the weighted sum of delay and energy consumption, it will produce uncertainty in the verification results. Because the delay is significantly larger than energy consumption in value, the performance of the large value will degrade the performance of the small value, which will have a certain impact on

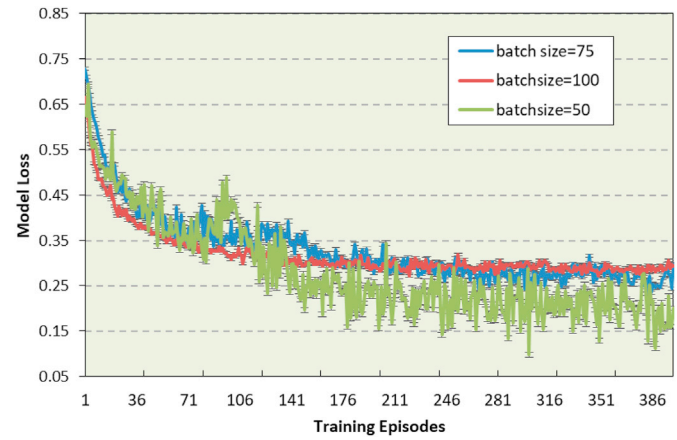

Fig. 9. Effect of model training batch data size on model loss.

the analysis and comparison of the results. This article considers the separation of time delay and energy consumption to verify the comparison. In addition, considering that millimeter waves in 5G will become a future trend, in some experiments, the performance of the algorithm is compared and verified on basis of the channel bandwidths of the two frequency bands in 5G.

Fig. 12 shows that energy consumption generated by FL is lower than that generated by No-FL. As the number of participating users increases, the energy consumption generated by the federal framework will be reduced, which can save 15%–35% of energy consumed, that is, the benefits of the federal framework increase. When using FL, in addition to the energy consumption advantage, for each participating user, it is more in line with the fact that the amount of user equipment data in real life is small, while also protecting the data privacy of the local user equipment and saving model training energy consumption.

Given the impact of channel bandwidth on energy consumption, in the experimental parameters set earlier in this article, the channel bandwidth is set to the channel bandwidths in the high frequency band and the low frequency band in 5G technology for performance comparison. The experimental results are shown in Fig. 13, where FL-sub6G and FL-mmWave represent the energy consumption generated by the algorithm in the low frequency band (sub6G) channel bandwidth and the energy consumption generated by the algorithm in the high frequency band (mmWave) channel bandwidth, respectively.

Fig. 13 shows that the energy consumption of FL-sub6G is higher than that of FL-mmWave, and the larger the number of participating users in the FL framework, the smaller the energy consumption gap between the two. According to the analysis of the results, due to the small number of participating users, the number of iterations will increase, and the different channel bandwidths of FL-sub6G and FL-mmWave will increase the energy consumption gap.

Next, the delay of the user equipment is verified. Similar to the verification above, it is still compared with the No-FL algorithm to verify the significance of the introduction of the FL in terms of delay.

Given the comparison of the delay caused by FL and No-FL on the decision-making model training, Fig. 14 shows that the delay produced by No-FL is smaller than produced by FL. The analysis of the result shows that in No-FL, no additional transmission delay is required, that is, the influence factors of the time-varying communication quality on the delay are not considered. The FL framework protects user data privacy and reduces energy consumption. However, the use of small dataset equipment will degrade delay performance as a price. However, it can be observed that when the number of participating users increases to 30, the delay caused by FL is slightly difference from the delay caused by No-FL. Compared with No-FL, an increase in the number of participating users will alleviate the delay impact caused by the FL.

Given the influence of channel bandwidth on delay, this study sets the

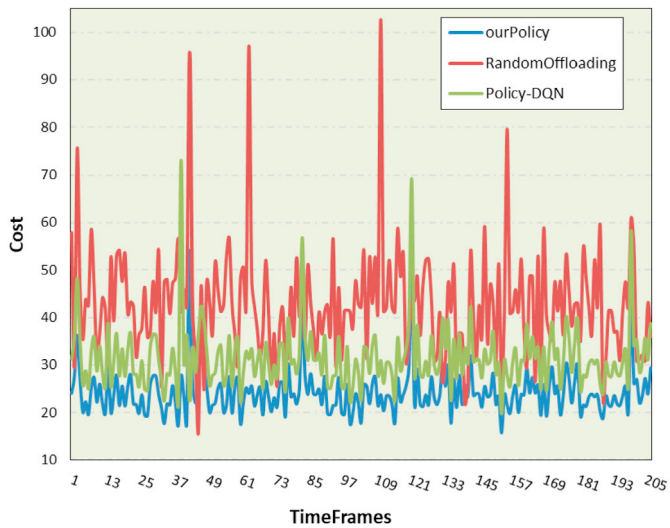

Fig. 10. The comparison of adaptive decision algorithms.

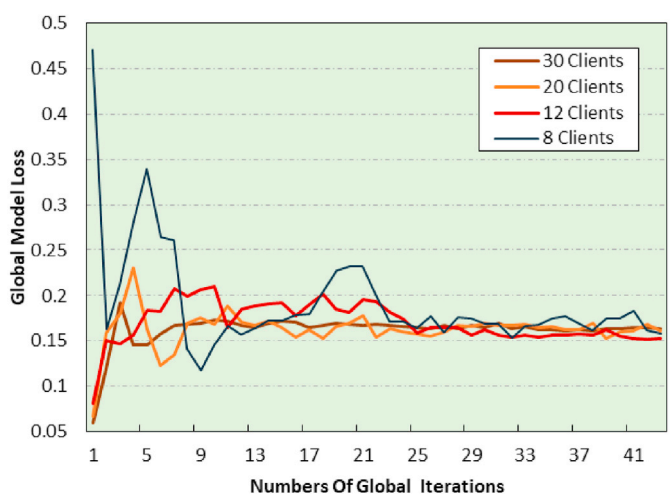

Fig. 11. Variations in the number of users involved in the global model loss.

channel bandwidth to the channel bandwidth of the two frequency bands in 5G technology in the original parameters for comparison and verification. The experimental results are shown in Fig. 15.

In Fig. 15, FL-sub6G and FL-mmWave respectively represent the delay generated by the algorithm under the channel bandwidth of the low frequency band and the delay generated using the algorithm under the channel bandwidth of the high frequency band. Fig. 15 shows that the delay generated by the low-frequency FL-sub6G is higher than that generated by the high-frequency FL-mmWave. According to this result, we posit that the channel bandwidth used by FL-sub6G is lower than that used by FL-mmWave, the transmission delay generated during global iteration will be affected by the channel bandwidth. The greater the number of participating users in the FL framework, the smaller the delay difference between the two delay generation processes.

In summary, the intelligent decision model optimization algorithm based on FL not only protects data privacy, increases the universality of the model, and reduces the "catastrophic forget" of the decision model, but also reduces the energy consumption of user equipment and improves the utility of FL. In the high frequency band of 5G, the utility of FL in the optimization algorithm of the intelligent decision model based on FL is more obvious.

In time-sensitive applications, our framework must protect privacy at the cost of high delay. In particular, in the case of fewer participating users, the delay is higher. In response to the above problems, this study designs a FL oriented communication delay optimization algorithm based on the contribution rate of unary outlier detection, and verifies the delay performance. The experimental results are shown in Fig. 16, FL, No-FL and FL-Update represents an algorithm that uses the federated framework, the algorithm that does not use the federated framework, and the communication delay optimization algorithm, respectively.

Fig. 16 shows that the overall delay can be significantly reduced by using the communication delay optimization algorithm. In the use of the communication delay optimization algorithm, an increase in the number of participating users will also reduce the delay. In a scenario with the same number of participating users, compared to FL and No-FL, the delay can be reduced by 15%–20% using the FL-Update algorithm. The greater the number of participating users, the more obvious the degree of delay reduction will be, and when the number of participating users increases to 30, the delay is lower than the delay caused by No-FL in the federal framework after applying the FL-Update algorithm.

Considering that the communication delay optimization algorithm is optimized for transmission delay, the difference in channel bandwidth may affect the delay of the federal framework. Therefore, this article continues to verify the influence of channel bandwidth on delay in the communication delay optimization algorithm. In the communication delay optimization algorithm, the channel bandwidth is set to the bandwidth under the high frequency band (mmWave) and the bandwidth under the low frequency band (Sub6G) in 5G for comparison and verification. The experimental results are shown in Fig. 17. FL-Update-sub6G in Fig. 17 represents the experimental results under sub6G. Similarly, FL-Update-mmWave represents the experimental results under mmWave.

It can be obtained from the experimental graph that the delay generated by FL-Update-sub6G is higher than the delay generated by FL-Update-mmWave. However, with an increase in the number of participating users, the difference between the two channel bandwidths in terms of delay. Given this situation, the analysis shows that because the more participating users, the faster the model converges, and the smaller the transmission delay. The bandwidth is mainly used for data transmission, therefore, the greater the number of participating users, the smaller the gap between the two channel bandwidths. In the communication delay optimization algorithm, the larger the channel bandwidth, the lower the time delay. In summary, the optimization of FL-Update can reduce the delay of the overall framework, and the delay optimization effect is more obvious under high channel bandwidth.

This study further investigates the influence of the contribution rate-based communication delay optimization algorithm on the energy consumption of the federated framework in reducing the delay, and compares it with the algorithm of the federated framework and the algorithm without the federated framework. The verification of energy consumption performance is shown in Fig. 18. FL, No-FL and FL-Update in Fig. 18

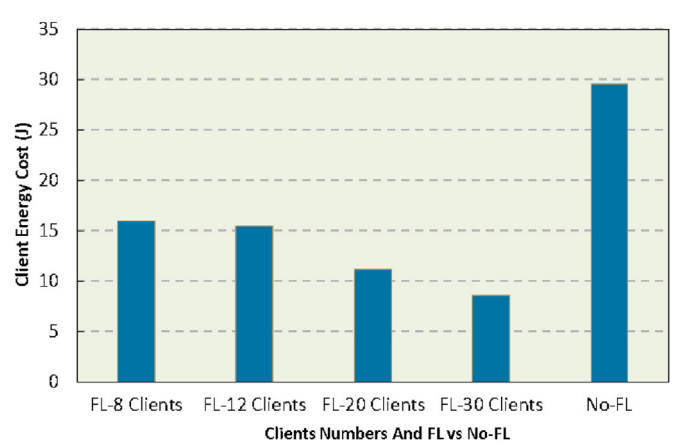

Fig. 12. Energy consumption for different number of users, with FL or No-FL.

Fig. 13. The effect of the number of users and channel bandwidth on the energy cost of FL framework.

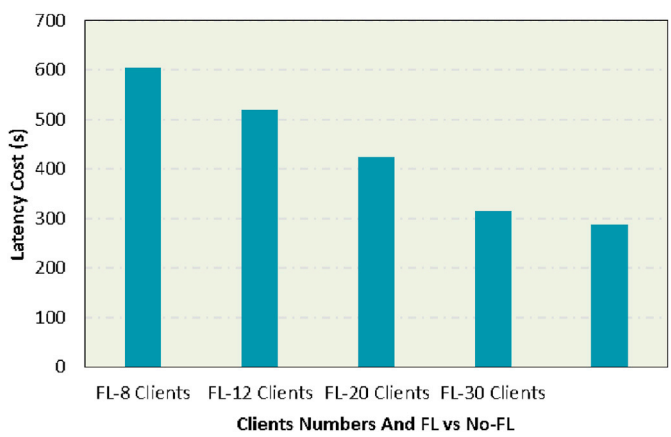

Fig. 14. The delays for different number of users, with FL or No-FL.

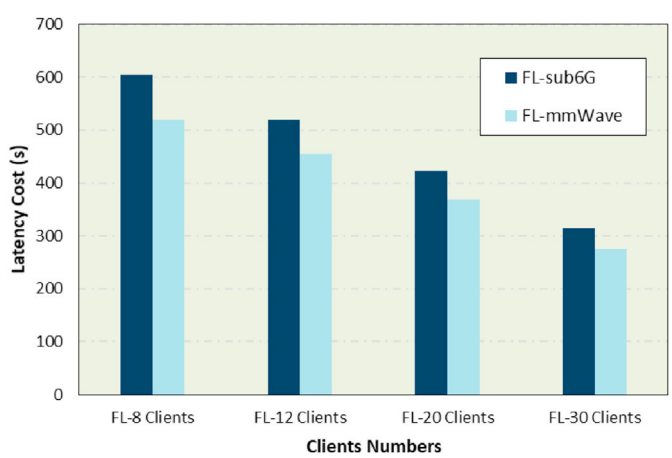

Fig. 15. Effect of the number of users and channel bandwidth on the delay of FL.

represent algorithm that use the federated framework, algorithm that does not use the federated framework, and algorithm that communication delay optimization, respectively.

As shown in Fig. 18, compared with FL, the proposed communication delay optimization algorithm can reduce the energy consumption by 10%–15%. Compared with NO-FL, the proposed communication delay optimization algorithm can reduce energy consumption by 30%–55%. Compared with NO-FL the results show that the communication delay optimization algorithm can increase the utility of the federated framework.

This study continues to study the influence of channel bandwidth on energy consumption in communication delay optimization algorithm. In the performance verification experiment, the channel bandwidth in the experiment is set to high frequency channel (mmWave) and low frequency channel (SUB6G) in 5G respectively for experimental comparison. The experimental results are shown in Fig. 19. FL-Update-sub6G in Fig. 19 represents the experimental results under sub6G. Similarly, FL-Update-mmWave represents the experimental results under mmWave.

It can be observed that the effect of channel bandwidth on energy consumption is similar to the effect of channel bandwidth on delay. The energy consumption of the communication delay optimization algorithm under mmWave is lower than that under SUB6G. However, as the number of users increases, the energy difference between the two will decrease.

To evaluate the effectiveness of the algorithm, we experimentally compared the proposed algorithm (MEModel) and three other offloading policy algorithms. The three algorithms were as follows: (i) AllLocal, where all tasks were calculated locally; (ii) AllServer, where all tasks were offloaded to the edge server for calculation; and (iii) Random-Offloading, where all tasks were calculated randomly on local or edge servers; (iv) DQNModel: FL is used for distributed training in the entire MEC architecture, and DQN algorithm is used on the local device to learn the decision model. When the model is stable, the computing task is decided and scheduled.

It can be observed from Fig. 20 that the cost of the AllLocal algorithm was the largest. The AllServer algorithm was less expensive for local users than the AllLocal algorithm; however, when task was transmitted to the edge server for offloading, the link quality could not be guaranteed, and transmission congestion was sometimes caused by the varying the link status, resulting in a substantial increase in transmission overhead. Although there was no local computational overhead, it was difficult to reduce the overall overhead due to the time-varying link state. The RandomOffloading algorithm used the combination of local and edge servers, which was similar to the algorithm proposed in this study; however, it did not systematically examine the overall status and offload accordingly. Therefore, unsatisfactory link quality sometimes caused the transmission calculation to be delayed, increasing the overall overhead. However, when the link quality was ideal, the combination of local and edge servers had better performance. Compared with the Random-Offloading algorithm, our offloading decision algorithm fully considered the local calculation situation and the quality of the link in making a decision, thereby providing an offloading scheme with the least overall overhead in the current state. The DQNModel is similar to the algorithm in this study, but because the model obtained by the DQN algorithm easily overestimate, the task scheduling decision given by the decision model can easily be biased, which increases the overall cost of computing tasks. Compared with DQNModel, the decision model obtained using the decision algorithm in this study make it easier to make appropriate computing task scheduling decisions based on the current computing environment state, which can minimise overhead costs and improve the quality of user experience. The experimental results thus demonstrate that compared to the other four offloading decision algorithms, our decision-making algorithm is optimal in terms of overhead.

6. Conclusion

In this study, in response to the current user requirements of low latency and high reliability, MEC is used to perform user task offloading. For time-varying link quality, we designed an adaptive task offloading decision algorithm based on DDQN. Under given channel conditions, corresponding task scheduling decisions are provided to reduce delay and energy consumption, thereby improving the quality of experience. Combined with FL, customers who do not have sufficient private data to develop accurate local task decision models can learn from collaboration. The benefits of the model are consistent with the actual application of

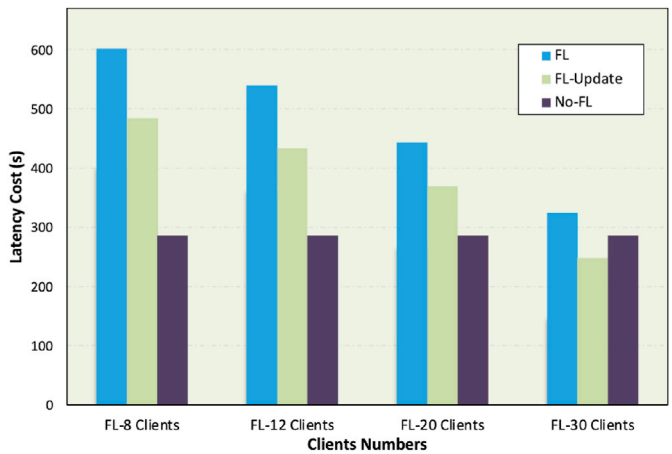

Fig. 16. The effect of communication delay optimization algorithm (FL-Update) on delay.

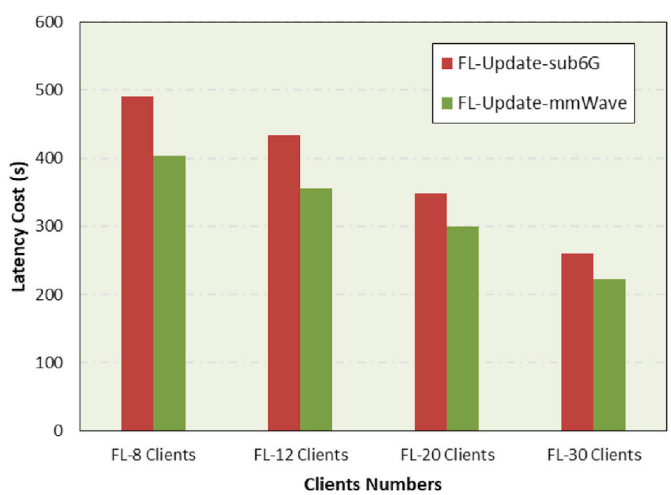

Fig. 17. The effect of the number of users and channel bandwidth on the delay of communication delay optimization algorithm (FL-Update).

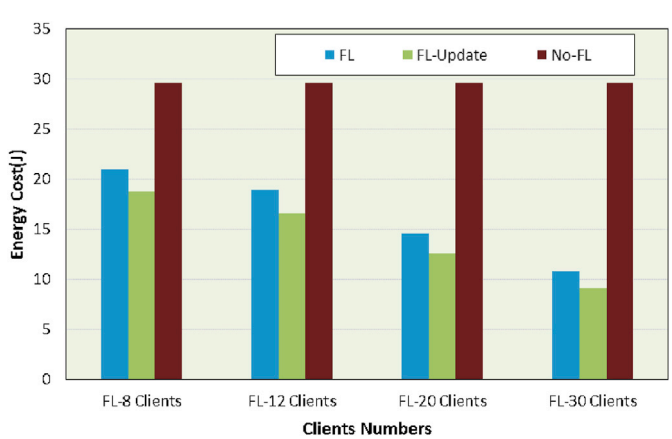

Fig. 18. The effect of communication delay optimization algorithm on energy cost.

users with little data on their devices and privacy concerns. Considering the resource consumption caused by FL, this study constructs a QoS model of federated framework based on time delay and energy

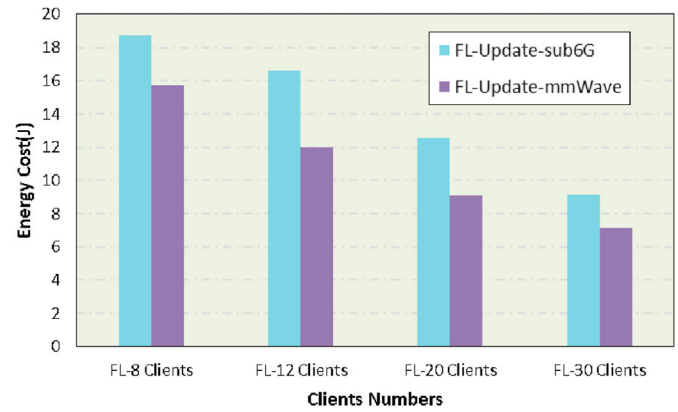

Fig. 19. The effect of number of users and channel bandwidth on the delay of communication delay optimization algorithm.

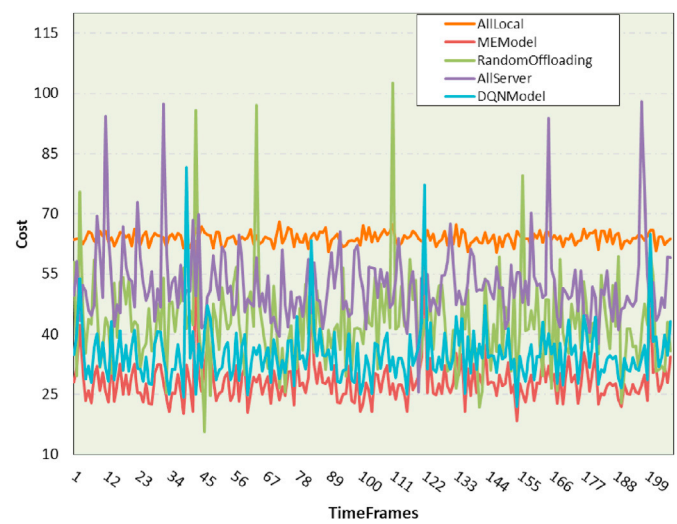

Fig. 20. Comparison between proposed algorithm (MEModel) and four other algorithms.

consumption, and uses this model to evaluate the utility of FL. By analyzing the calculation model of the communication delay of FL, this paper designs a communication delay optimization algorithm based on the contribution rate for FL. This algorithm optimizes the delay generated by FL, reduces the resource consumption caused by FL, and improves the utility of introducing FL. Through comparison with several benchmark solutions, we evaluated the performance of the proposed solution, and the simulation results revealed that the proposed scheme had better performance.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is supported by the National Natural Science Foundation of China(62032013,62072094; Liaoning Province Science and Technology Fund Project(2020MS086); Shenyang Science and Technology Plan Project(20206424); the Fundamental Research Funds for the Central Universities(N2116014,N180101028) CERNET Innovation Project (NGII20190504).

References

[1] Cisco, Cisco Visual Networking Index: Global Mobile Data Traffic Forecast Update, 2018–2023 White Paper, 2020. https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper- c11-741490.html. (Accessed 15 December 2020).

[2] ITU-R, Imt Vision-Framework and Overall Objectives of the Future Development of Imt for 2020 and beyond, 2015. https://www.itu.int/dms-pubrec/itu-r/rec/m/R-REC-M.2083-0-201509-I!!PDF-E.pdf. (Accessed 15 December 2020).

[3] D. Demmer, R. Gerzaguet, J.-B. Doré, D. Le Ruyet, Analytical study of 5g nr embb co-existence, in: 2018 25th International Conference on Telecommunications (ICT), IEEE, 2018, pp. 186–190.

[4] C. Bockelmann, N. Pratas, H. Nikopour, K. Au, T. Svensson, C. Stefanovic, P. Popovski, A. Dekorsy, Massive machine-type communications in 5g: physical and mac-layer solutions, IEEE Commun. Mag. 54 (9) (2016) 59–65.

[5] G. Pocovi, H. Shariatmadari, G. Berardinelli, K. Pedersen, J. Steiner, Z. Li, Achieving ultra-reliable low-latency communications: challenges and envisioned system enhancements, IEEE Network 32 (2) (2018) 8–15.

[6] Y. Yu, 5G Mobile Edge Computing, People's Posts and Telecommunications Press, Beijing, 2017.

[7] J. Ren, D. Zhang, S. He, Y. Zhang, T. Li, A survey on end-edge-cloud orchestrated network computing paradigms: transparent computing, mobile edge computing, fog computing, and cloudlet, ACM Comput. Surv. 52 (6) (2019) 1–36.

[8] N. Abbas, Y. Zhang, A. Taherkordi, T. Skeie, Mobile edge computing: a survey, IEEE Internet Things J. 5 (1) (2018) 450–465.

[9] S. Xia, Z. Yao, Y. Li, S. Mao, Online distributed offloading and computing resource management with energy harvesting for heterogeneous mec-enabled iot, IEEE Trans. Wireless Commun. 20 (10) (2021) 6743–6757.

[10] G. Gao, Y. Wen, Video transcoding for adaptive bitrate streaming over edge-cloud continuum, Digit. Commun. Netw. 7 (4) (2021) 598–604.

[11] Y. Li, H. Ma, L. Wang, S. Mao, G. Wang, Optimized content caching and user association for edge computing in densely deployed heterogeneous networks, IEEE Trans. Mobile Comput. 7 (99) (2020) 1–13.

[12] H. Liu, L. Cao, T. Pei, Q. Deng, J. Zhu, A fast algorithm for energy-saving offloading with reliability and latency requirements in multi-access edge computing, 2019, IEEE Access 8 (2019) 151–161.

[13] Y. Guo, Z. Zhao, R. Zhao, S. Lai, Z. Dan, J. Xia, L. Fan, Intelligent offloading strategy design for relaying mobile edge computing networks, 2020, IEEE Access 8 (2020) 35127–35135.

[14] Y. Sun, T. Wei, H. Li, Y. Zhang, W. Wu, Energy-efficient multimedia task assignment and computing offloading for mobile edge computing networks, 2020, IEEE Access 8 (2020) 36702–36713.

[15] P. Sun, H. Zhang, H. Ji, X. Li, Task allocation for multi-aps with mobile edge computing, in: 2018 IEEE/CIC International Conference on Communications in China (ICCC Workshops), IEEE, 2018, pp. 314–318.

[16] M. Avgeris, D. Dechouniotis, N. Athanasopoulos, S. Papavassiliou, Adaptive resource allocation for computation offloading: a control-theoretic approach, ACM Trans. Internet Technol. 19 (2) (2019) 1–20.

[17] S. Jošilo, G. Dán, Wireless and computing resource allocation for selfish computation offloading in edge computing, in: IEEE INFOCOM 2019-IEEE Conference on Computer Communications, IEEE, 2019, pp. 2467–2475.

[18] J. Meng, H. Tan, C. Xu, W. Cao, L. Liu, B. Li, Dedas: online task dispatching and scheduling with bandwidth constraint in edge computing, in: IEEE INFOCOM 2019-IEEE Conference on Computer Communications, IEEE, 2019, pp. 2287–2295.

[19] Y. Qu, J. Liu, Computation offloading for mobile edge computing with accuracy guarantee, in: Proceedings of the ACM Turing Celebration Conference-China, ACM, 2019, pp. 1–5.

[20] S. Pasteris, S. Wang, M. Herbster, T. He, Service placement with provable guarantees in heterogeneous edge computing systems, in: IEEE INFOCOM 2019-IEEE Conference on Computer Communications, IEEE, 2019, pp. 514–522.

[21] S. Sundar, B. Liang, Offloading dependent tasks with communication delay and deadline constraint, in: IEEE INFOCOM 2018-IEEE Conference on Computer Communications, IEEE, 2018, pp. 37–45.

[22] T. Alfakih, M.M. Hassan, A. Gumaei, C. Savaglio, G. Fortino, Task offloading and resource allocation for mobile edge computing by deep reinforcement learning based on sarsa, 2020, IEEE Access 8 (2020) 54074–54084.

[23] E. Biglieri, J. Proakis, S. Shamai, Fading channels: information-theoretic and communications aspects, IEEE Trans. Inf. Theor. 44 (6) (1998) 2619–2692.

[24] H. Li, N. Kumar, R. Chen, P. Georgiou, A deep reinforcement learning framework for identifying funny scenes in movies, in: 2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, 2018, pp. 3116–3120.

[25] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M. Riedmiller, Playing Atari with Deep Reinforcement Learning, 2013. https://www.cs.toronto.edu/vmnih/docs/dqn.pdf. (Accessed 15 January 2021).

[26] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, et al., Human-level control through deep reinforcement learning, Nature 518 (7540) (2015) 529–533.

[27] Y. Li, Deep Reinforcement Learning: an Overview, 2017. https://arxiv.org/abs/1701.07274. (Accessed 19 January 2021).

[28] Q. Yang, Y. Liu, Y. Cheng, Y. Kang, T. Chen, H. Yu, Federated learning, 3, in: Synthesis Lectures on Artificial Intelligence and Machine Learning 13, 2019, 1–207.

[29] G.R. Blog, Federated Learning: Collaborative Machine Learning without Centralized Training Data, 2017. https://www.googblogs.com/federated-learning-collaborative-machine-learning-without-centralized-training-data/. (Accessed 26 December 2020).

[30] R. Li, F. Ma, W. Jiang, J. Gao, Online federated multitask learning, in: 2019 IEEE International Conference on Big Data (Big Data), IEEE, 2019, pp. 215–220.

[31] C.W. Zaw, C.S. Hong, Loss and energy tradeoff in multi-access edge computing enabled federated learning, in: 2021 International Conference on Information Networking (ICOIN), IEEE, 2021, pp. 597–602.

[32] B. Yang, X. Cao, X. Li, T. Kroecker, L. Qian, Joint communication and computing optimization for hierarchical machine learning tasks distribution, in: 2019 IEEE Symposium on Computers and Communications (ISCC), IEEE, 2019, pp. 1–6.

[33] T. Braud, Z. Pengyuan, J. Kangasharju, H. Pan, Multipath computation offloading for mobile augmented reality, in: 2020 IEEE International Conference on Pervasive Computing and Communications (PerCom), IEEE, 2020, pp. 1–10.

[34] T. Verbelen, P. Simoens, F. De Turck, B. Dhoedt, Cloudlets: bringing the cloud to the mobile user, in: Proceedings of the Third ACM Workshop on Mobile Cloud Computing and Services, ACM, 2012, pp. 29–36.

[35] Y. Hao, M. Chen, L. Hu, M.S. Hossain, A. Ghoneim, Energy efficient task caching and offloading for mobile edge computing, 2018, IEEE Access 6 (2018) 11365–11373.

[36] K. Wang, Z. Hu, Q. Ai, Y. Zhong, J. Yu, P. Zhou, L. Chen, H. Shin, Joint offloading and charge cost minimization in mobile edge computing, 2020, IEEE Open J. Commun. Soc. 1 (2020) 205–216.

[37] P. Yao, X. Chen, Y. Chen, Z. Li, Deep reinforcement learning based offloading scheme for mobile edge computing, in: 2019 IEEE International Conference on Smart Internet of Things (SmartIoT), IEEE, 2019, pp. 417–421.

[38] S. Pang, S. Wang, Joint wireless source management and task offloading in ultra-dense network, 2020, IEEE Access 8 (2020) 52917–52926.